TECNICATURA UNIVERSITARIA EN DESARROLLO WEB · FCAD · UNER

Unidad 2
Testing

Desarrollo para Móviles · 2026 — Segundo cuatrimestre

Prof. Titular: Lic. Franco Ruis Dias · JTP: Lic. Germán Cueva

Expo SDK 57 · Jest · Testing Library · Maestro

UNIDAD 2

Qué vamos a ver

Objetivos

- Saber qué conviene testear y qué no.
- Escribir el primer test de un componente.
- Conocer las tres escalas de prueba.

Temas

- La pirámide: unidad, integración, E2E
- `jest-expo` y Jest
- **React Native Testing Library**
- Expo Router · Maestro · `expo lint`

> La clase anterior fue sobre encontrar un error. Ésta es sobre **que no vuelva**.

UNIDAD 2 · PARTE 1

Qué se testea

Y, sobre todo, qué no.

Para qué sirve un test

Es una red

Podés cambiar código con la confianza de que si rompiste algo, te vas a enterar.

Es documentación

Un test dice qué se espera que haga la función, con ejemplos concretos. Y no se desactualiza sin avisar.

Evita la repetición

Un bug que ya apareció y tiene test no vuelve. Sin test, vuelve.

> Lo que un test **no** es: una garantía de que la app funciona. Cobertura del 100 % con tests que no prueban nada real es peor que no tener, porque da tranquilidad falsa.

Las tres escalas

	Qué prueba	Con qué	Costo
Unidad	Una función o un componente, aislado	Jest + Testing Library	Barato y rápido
Integración	Varias piezas juntas: una pantalla con su navegación y sus datos	`expo-router/testing-library`	Medio
E2E	La app entera, instalada, como la usa una persona	Maestro	Caro y lento

> Muchos de los primeros, algunos de los segundos, pocos de los terceros. Los E2E son los que más confianza dan y los que más cuestan de mantener: se rompen cuando cambiás un texto de la interfaz.

Qué conviene testear en el TP

Sí

- Lógica propia: cálculos, validaciones, transformaciones de los datos de la API.
- Componentes con condiciones: qué muestra si la lista está vacía, si hay error.
- Lo que ya se rompió una vez.

No

- Que React Native renderice un `<Text>`. Eso ya está testeado.
- Detalles de implementación: nombres de estado interno, cuántas veces se llamó a una función privada.
- Estilos exactos. Cambian todo el tiempo y no son el comportamiento.

> El criterio que ordena: **testeá lo que el usuario percibe**, no cómo está hecho por dentro. Así el test sobrevive a una refactorización.

UNIDAD 2 · PARTE 2

Jest y `jest-expo`

El primer test, de punta a punta.

Instalación

```
npx expo install jest-expo jest @types/jest --dev
npx expo install @testing-library/react-native --dev
```

package.json

```
{
  "scripts": {
    "test": "jest --watchAll"
  },
  "jest": { "preset": "jest-expo" }
}
```

tsconfig.json

```
{
  "compilerOptions": {
    "types": ["jest"]
  }
}
```

> `jest-expo` es un preset: simula la parte nativa del SDK de Expo y deja casi toda la configuración resuelta. Sin él, importar cualquier módulo de Expo haría explotar el test. Y sin `"types": ["jest"]`, TypeScript no conoce `test` ni `expect` y el editor los marca en rojo aunque los tests pasen.

El primer test

EL COMPONENTE

```
// src/app/index.tsx
export default function Inicio() {
  return (
    <View>
      <Text>¡Bienvenida!</Text>
    </View>
  );
}
```

EL TEST

```
// __tests__/inicio-test.tsx
import { render, screen }
  from '@testing-library/react-native';
import Inicio from '@/app/index';

test('saluda al abrir', () => {
  render(<Inicio />);
  expect(screen.getByText('¡Bienvenida!'))
    .toBeVisible();
});
```

El preset reconoce como test los archivos que terminan en `-test.ts` / `-test.tsx` , y los que están dentro de `__tests__` .

Cómo organizar los tests

UN `__tests__` EN LA RAÍZ

```
__tests__/
  inicio-test.tsx
  tarjeta-test.tsx
src/
  app/
    index.tsx
  componentes/
    tarjeta.tsx
```

Alcanza mientras haya pocos tests. Todo junto se encuentra rápido.

UNO POR ÁREA, AL LADO DEL CÓDIGO

```
src/
  app/
    index.tsx
  componentes/
    tarjeta.tsx
    __tests__/
      tarjeta-test.tsx
  utils/
    precios.ts
    __tests__/
      precios-test.ts
```

El test viaja con el archivo que prueba: si se mueve la carpeta, se mueve todo junto.

Dos reglas. Una: dentro de `app/` **no** van tests —ahí todo tiene que ser una ruta—, así que las pantallas se prueban siempre desde el `__tests__` de la raíz. Dos: ahí conviene nombrar por el **componente** (`inicio-test.tsx`), porque con Expo Router todas las rutas se llaman `index.tsx` y `index-test.tsx` no diría cuál es.

Buscar como busca un usuario

```
screen.getByText('Guardar');                    // por el texto visible
screen.getByLabelText('Cerrar');                // por la etiqueta de accesibilidad
screen.getByPlaceholderText('Buscá un producto');
screen.getByRole('button', { name: 'Enviar' });
```

`getBy…`

Tiene que existir. Si no, el test falla.

`queryBy…`

Devuelve `null` si no está. Es el que se usa para afirmar que algo no aparece.

`findBy…`

Espera a que aparezca. Para lo que llega de forma asíncrona.

> Fijate que ninguna de estas consultas menciona clases ni estructura interna. Buscan por lo que se ve — que es justamente lo que hace que el test no se rompa al refactorizar.

Interacción y espera

```
import { render, screen, userEvent } from '@testing-library/react-native';

test('muestra el error si el campo está vacío', async () => {
  const usuario = userEvent.setup();
  render(<Formulario />);

  await usuario.press(screen.getByText('Enviar'));

  expect(await screen.findByText('El nombre es obligatorio')).toBeVisible();
});
```

> Tres pasos, siempre los mismos: **preparar** (render), **actuar** (press, type) y **verificar** (expect). Si un test tuyo no se lee así, probablemente esté probando de más.

Tests de instantánea

```
test('la tarjeta se ve igual', () => {
  const arbol = render(<Tarjeta producto={ejemplo} />).toJSON();
  expect(arbol).toMatchSnapshot();
});
```

Qué hacen

Guardan el árbol renderizado en un archivo y avisan si cambia. Se actualizan con `jest -u`.

Por qué usarlos poco

Fallan ante cualquier cambio legítimo, y la reacción natural es actualizarlos sin mirar. Cuando eso pasa, dejaron de servir. **Expo recomienda E2E antes que instantáneas** para probar interfaz.

Correr los tests y medir cobertura

FLUJOS, EN `PACKAGE.JSON`

```
"scripts": {
  "test": "jest --watch --changedSince=origin/main",
  "test:final": "jest",
  "test:snap": "jest -u"
}
```

`test` mientras programás: queda corriendo y sólo prueba lo que cambiaste respecto de `main`. `test:final` antes de entregar, corre todo. `test:snap` regenera las instantáneas.

COBERTURA, EN LA CLAVE `JEST`

```
"jest": {
  "preset": "jest-expo",
  "collectCoverage": true,
  "collectCoverageFrom": [
    "**/*.{ts,tsx}",
    "!**/coverage/**",
    "!**/node_modules/**",
    "!**/.expo/**"
  ]
}
```

Dónde ver el reporte

Queda en `coverage/lcov-report/index.html`: se abre en el navegador y muestra línea por línea qué se ejecutó. `coverage/**/*` va al `.gitignore`.

Para qué NO sirve

Como número a maximizar. Sirve para lo contrario: encontrar el archivo que nadie probó nunca. Un 90% de cobertura con tests que no verifican nada da exactamente cero confianza.

Un aviso sobre material viejo

`react-test-renderer` DEPRECADO

Es lo que usan casi todos los tutoriales de testing en React Native anteriores a 2025. **No soporta React 19**, así que con nuestro SDK no sirve.

`@testing-library/react-native`

Es el reemplazo, y es lo que recomienda la documentación de Expo. Si tenías el otro instalado, sacalo.

> Es el mismo patrón que ya vimos con Flipper y con `ANDROID_SDK_ROOT` : el ecosistema se mueve rápido y el primer resultado de búsqueda suele ir un par de años atrás. Verificar contra la documentación es parte del oficio.

UNIDAD 2 · PARTE 3

Integración y E2E

Cuando hace falta probar más de una pieza.

Probar la navegación

```
import { renderRouter, screen } from 'expo-router/testing-library';

test('el detalle abre en la ruta correcta', () => {
  renderRouter(['index', 'productos/[id]'], { initialUrl: '/productos/12' });

  expect(screen).toHavePathname('/productos/12');
});
```

Expo Router arma las rutas leyendo el sistema de archivos, así que testearlo requiere simular ese sistema. `expo-router/testing-library` lo hace: le pasás una lista de rutas y te arma una app en memoria.

Los archivos de test **no van dentro de** `app/` : ahí todo tiene que ser una ruta o un layout. Van en `__tests__` .

E2E con Maestro

```yaml
# .maestro/inicio.yml
appId: ar.edu.uner.miapp
---
- launchApp
- assertVisible: '¡Bienvenida!'
- tapOn: 'Explorar'
- assertVisible: 'Productos.*'
```

Qué prueba

La app instalada de verdad, en un emulador o un dispositivo, tocando la pantalla como una persona.

Cómo se corre

`maestro test .maestro/inicio.yml` en tu máquina, o en la nube dentro de un flujo de EAS Workflows, disparado por cada Pull Request.

> Los flujos se escriben en YAML, no en JavaScript. Es a propósito: son cortos y los puede leer alguien que no programa.

`expo lint`: el test que no escribís

```
npx expo lint
```

```
/src/components/hello-wave.tsx
  22:6  warning  React Hook useEffect has a missing dependency: 'rotateAnimation'
                 react-hooks/exhaustive-deps
✖ 1 problem (0 errors, 1 warning)
```

La primera vez que lo corrés, crea el `eslint.config.js` con la configuración de Expo. Y esa advertencia de arriba es exactamente el error de dependencias que vimos en la clase de ciclo de vida: **el linter lo encuentra solo**.

Sumado a **TypeScript**, que ya verifica los tipos, tenés dos capas de control que no cuestan escribir un solo test.

TDD: escribir el test primero

TDD —Test-Driven Development, desarrollo guiado por pruebas— da vuelta el orden de siempre: en lugar de escribir el código y después probarlo, el test va primero y es el que **define** qué tiene que hacer el código. Lo que sigue tiene sentido sólo si trabajás así.

1 · Rojo

Escribís el test de algo que todavía no existe. Lo corrés y **falla**. Eso confirma que el test realmente prueba algo.

2 · Verde

Escribís el código mínimo que lo hace pasar. Sin adornos ni casos que nadie pidió.

3 · Refactor

Ahora sí lo ordenás. El test te avisa en el acto si en el camino rompiste el comportamiento.

Dónde rinde

Lógica pura: un cálculo, una validación, transformar la respuesta de la API. Sabés qué tiene que dar, así que podés escribirlo antes.

Dónde estorba

La interfaz. No sabés cómo va a quedar hasta verla: escribir el test antes es adivinar.

Y con agentes

Si el código lo escribe una IA, el test es lo que le **define** qué pedís y lo que **verifica** que lo hizo.

> El caso donde conviene siempre, aunque no hagan TDD para nada más: **cuando aparece un bug**. Antes de tocar el código, escribí el test que lo reproduce y miralo fallar. Si no falla, todavía no entendiste el problema.

Por dónde empezar

1
`expo lint` y TypeScript sin errores

2
Tests de la lógica propia: funciones puras

3
Un test por cada componente con condiciones

4
Un E2E del camino principal de la app

> Para el trabajo integrador, con los pasos 1 y 2 ya cambia bastante la calidad de la entrega. Nadie espera cobertura total: se espera que lo que es fácil de romper esté cubierto.

UNIDAD 2 · DOCUMENTACIÓN

Documentación

Todo lo de esta clase sale de acá.

Fuentes

Testing unitario con Jest La guía de Expo: instalación, estructura, cobertura	`docs.expo.dev/develop/unit-testing`
React Native Testing Library Consultas, `userEvent`, buenas prácticas	`callstack.github.io/react-native-testing-library`
Testing de Expo Router `renderRouter` y sus matchers	`docs.expo.dev/router/reference/testing`
E2E con Maestro en EAS	`docs.expo.dev/eas/workflows/examples/e2e-tests`
Maestro Referencia de comandos de los flujos	`docs.maestro.dev`
Jest	`jestjs.io/docs/getting-started`
ESLint y Prettier en Expo	`docs.expo.dev/guides/using-eslint`

UNIDAD 2

Para llevarse

- Testeá **lo que el usuario percibe**, no cómo está hecho por dentro.
- Muchos tests de unidad, algunos de integración, **pocos E2E**.
- `expo lint` y TypeScript son la primera línea, y son gratis.
- Un bug con test no vuelve. Sin test, vuelve.